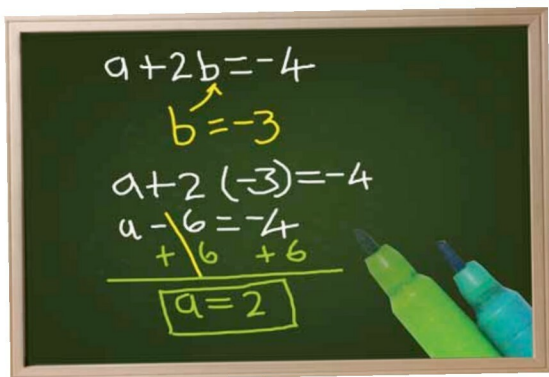


Learn How to Solve Non-Linear Equations **with Linear Algebra**

In this mathematical journey through open source, we solve non-linear equations using linear algebra in Octave.



I hope you found out the vegetable prices from the vendor, who had various equally priced stacks for sale at ₹ 30. Recall: One stack had 4 lemons, 7 cucumbers, and 9 tomatoes. Another had 2 lemons, 5 cucumbers, and 27 tomatoes. And the third had just 9 cucumbers and 15 tomatoes. The prices worked out at ₹ 2.00 per lemon, ₹ 2.50 per cucumber, and ₹ 0.50 per tomato, computed as follows:

```
$ octave -qf
octave:1> N = [
> 4 7 9
> 2 5 27
> 0 9 15
> ];
octave:2> inv(N) * [30; 30; 30]
ans =

2.00000
2.50000
0.50000

octave:3>
```

Polynomial solving

Note that this is a linear equation, though in three variables. How about solving higher order polynomial equations involving squares, cubes, etc, of the variable? Let's assume we want a solution for x in $x^3 + 3x^2 + 3x + 1 = 0$. Simple! First define a function for this polynomial. And then use the function solver *fsolve* to solve it, as follows:

```
$ octave -qf
octave:1> function y = f(x)
> y = x^3 + 3*x^2 + 3*x + 1;
> endfunction
octave:2> [x, fval, info] = fsolve(@f, 0)
x = -0.99999
fval = 0
info = 1
octave:3>
```

This indicates the value of x as $-0.99999 \approx -1$ as the solution to the function $f(x)$, yielding a function value of 0, with $\text{info} = 1$ indicating that the solution has been obtained. You may verify the answer by calling the function 'f' with the variable 'x' as $f(x)$ on the Octave prompt. The second